WhatsApp Auto-Reply with n8n: Complete Setup Guide

A step-by-step checklist for setting up a WhatsApp auto-reply (with optional AI) for each new client. Follow it in order and don't skip steps.

0. How it works (read once)

- **Receiving:** someone messages the business number. Meta sends a request (a webhook) to a public HTTPS address. n8n's **WhatsApp Trigger** catches it.
- **Replying:** n8n's **WhatsApp Business Cloud → Send message** node calls Meta's API with an access token.
- **The public address problem:** Meta can't reach `localhost`. A local n8n needs a tunnel (ngrok). A cloud-hosted n8n already has a public address.
- **Two different credentials in n8n:**
  - **WhatsApp OAuth API** (used by the Trigger): needs the Meta **App ID** and **App Secret**.
  - **WhatsApp API** (used by Send message): needs an **Access Token** and the **WhatsApp Business Account ID**.
- **Golden rules learned the hard way:**
  1. Meta allows only **one WhatsApp Trigger per Meta app**. A new client needs a **new Meta app**.
  2. The business number must **not** be active on a normal WhatsApp or WhatsApp Business app.
  3. Never click **Execute workflow** while the workflow is **Published**.
  4. Never paste tokens, App Secrets or ngrok authtokens into chats, emails or screenshots.

1. What you need before starting (per client)

- A Facebook account that will own the Meta developer app (ideally the client's business admin, or yours with the client added later).
- The client's **business phone number**. It must receive an SMS or voice call, and must be free (see section 4).

- The client's business name, a website or any public page URL (used as the privacy policy URL).
- An AI key if you want AI replies (for example a Google Gemini credential in n8n).
- n8n running with a public HTTPS address (section 7 or section 8).

2. Create the Meta app and business portfolio

1. Open **https://developers.facebook.com/apps/** and log in. Register as a developer if asked (verify phone or email, accept terms).
2. Click **Create App**. Enter an app name (for example `ClientName-n8n`) and your email.
3. If asked about a business portfolio, create or select the client's portfolio. If you hit the screen "Unable to access Meta Business Suite", don't use that path. Open **https://business.facebook.com/overview** and click **Create a business portfolio** instead (no Facebook Page needed).
4. Use case: **Other**, app type: **Business**. If you see "Connect with customers through WhatsApp", choose that.
5. Click **Create app**. On the dashboard, find **WhatsApp** and click **Set up**.
6. If the page says "Create a business portfolio", click **Continue**, create it, return to the app and refresh.
7. Write down the **App ID**.

3. Basic app settings and Live mode

1. Go to **App settings → Basic**.
2. Click **Show** next to **App Secret** and copy it somewhere private. You need it for n8n.
3. Fill in **Privacy policy URL** (any working public page) and choose a **Category** (for example Business and Pages). Click **Save changes**.
4. Switch **App Mode** from **Development** to **Live** (toggle at the top of the page). Refresh with F5 and confirm the bar says **Live**.
   - Why this matters: in Development mode Meta may deliver messages only for people connected to the app, so some customers get no reply.
5. Go to **App settings → Advanced → Security** and check that **Allow API Access to app settings** is **ON**. n8n needs it to register the webhook.

4. Free the client's phone number

The number can't be registered on a regular WhatsApp or WhatsApp Business app.

1. On the phone using that number, open WhatsApp (or WhatsApp Business).
2. Go to **Settings → Account → Delete account**. Uninstalling or logging out is not enough.
3. Check that the number isn't also logged in on another phone, a dual or clone app, or WhatsApp Web.
4. Wait 10 to 15 minutes. Never register that number in WhatsApp again after deleting, or it will be taken again.
5. If Meta later says "already registered", wait longer (up to a day) and try the **Phone call** verification option instead of text.

5. Register the number in Meta

1. In the app, open **WhatsApp → Step 2. Production setup**.
2. Work through the list: select or create the WhatsApp Business Account, enter the business display name and category, enter the phone number with country code, and verify with the 6-digit code (text or call).
3. Add a **payment method** if Meta asks.
4. When finished, open **WhatsApp Manager**: **https://business.facebook.com/latest/whatsapp_manager/phone_numbers**. Select the client's business at the top left.
5. Check that the number shows **Connected**.
6. Write down these three values:
   - **Phone Number ID** (open the number's details).
   - **WhatsApp Business Account ID** (WABA ID). It also appears in the browser address as `asset_id=...` on the WhatsApp Manager Overview page.
   - The display number itself.

Note: the **Step 1. Try it out** page only shows Meta's free **test number**. Your real number won't appear there, and that's normal. Use the test number only for practice (it can message at most 5 verified recipients).

6. Create a permanent access token (do this once per client)

The token from **Generate token** on the Try it out page expires in 24 hours. Use a system user token instead.

1. Open **https://business.facebook.com/settings** and select the client's business portfolio at the top.
2. Go to **Users → System users → Add**. Name it `n8n`, role **Admin**, then **Create system user**.
3. Select the system user and click **Assign assets**:
   - **WhatsApp accounts:** tick the client's WhatsApp account (and the test account if you want it), turn on **Full control**, click **Assign assets**.
   - **Apps:** tick the Meta app, turn on **Full control**, click **Assign assets**.
   - If you see "Unable to assign assets / all admins must confirm", open the developer site, check **App roles → Roles** for pending invites, open **Required actions**, fill **App settings → Basic**, wait a few hours and retry.
4. Click **Generate token**. App: your app. Token expiration: **Never**. Permissions: `whatsapp_business_messaging` and `whatsapp_business_management`.
5. Click **Generate token** and **copy it immediately**. Meta shows it only once. Store it in a password manager.
6. If a token ever leaks (chat, screenshot), go back to the system user, click **Revoke tokens**, generate a new one and update n8n.

7. Run n8n with a public address (local PC with ngrok)

Skip this section if n8n is hosted on a server or n8n Cloud (see section 8).

One-time ngrok setup

1. Create a free account at **ngrok.com**.
2. Install it in PowerShell: `winget install ngrok.ngrok`. Close and reopen PowerShell.
3. In the ngrok dashboard, open **Your Authtoken**, copy it and run:

```
ngrok config add-authtoken YOUR_TOKEN
```

4. In the dashboard, go to **Domains** and claim your free static domain (looks like `something.ngrok-free.dev`).

Every time you start (two PowerShell windows, in this order)

Window 1, the tunnel (use your own domain, port 5678):

```
ngrok http --url=https://YOUR-DOMAIN.ngrok-free.dev 5678
```

Wait for **Session Status: online**. Leave the window open.

Window 2, n8n (set the variable first, with the `/` at the end):

```
$env:WEBHOOK_URL="https://YOUR-DOMAIN.ngrok-free.dev/"
n8n
```

Then open n8n in the browser using **https://YOUR-DOMAIN.ngrok-free.dev** (not localhost). Click **Visit Site** on ngrok's warning page if shown.

Checks

- In the Trigger node, **Webhook URLs** must start with your ngrok address, not `http://localhost`. If it says localhost, `WEBHOOK_URL` wasn't set in that window. Stop n8n (Ctrl + C) and start again as above.
- To run n8n locally again without ngrok: `Remove-Item Env:WEBHOOK_URL` then `n8n`, and open `http://localhost:5678`. WhatsApp auto-reply doesn't work in that mode.
- The ngrok inspector at `http://127.0.0.1:4040` shows every request. A message from a customer appears there as `POST /webhook/...`.
- The `n8n start --tunnel` option no longer works in current n8n versions. Don't use it.
- Free ngrok gives one static domain. One n8n and one domain can serve several clients, because each workflow gets its own unique webhook path.

8. Better for real clients: hosted n8n

Your PC and ngrok must stay on for auto-reply to work. For paying clients, move to either:

- **n8n Cloud** (n8n.io): public HTTPS address built in, nothing to install.
- **A small VPS with Docker** and a domain with HTTPS.

You can export a workflow from n8n and import it on the new server. Credentials must be created again there.

9. Create the n8n credentials

Credential A: WhatsApp OAuth API (for the Trigger)

1. In n8n, open the WhatsApp Trigger node and choose **Create new credential**: **WhatsApp OAuth API**.
2. **Client ID:** the Meta App ID.
3. **Client Secret:** the App Secret (App settings → Basic → Show).
4. Click **Save**.

Credential B: WhatsApp API (for Send message)

1. In the Send message node, choose **Create new credential**: **WhatsApp API** (not OAuth).
2. **Access Token:** the permanent system user token from section 6.
3. **Business Account ID:** the WABA ID from section 5.
4. Click **Save**. A green success message should appear.

Name each credential after the client (for example `ClientName OAuth`) so they don't get mixed up. Mixing IDs from two different apps causes 401 and "Invalid parameter" errors.

10. Build the workflow

Flow: **WhatsApp Trigger → If → AI Agent → HTTP Request**. The HTTP Request node sends the reply because it works for every customer, including those with a WhatsApp username (see Node 4 and section 17).

Node 1: WhatsApp Trigger

- Credential: Credential A. **Trigger On:** Messages.

Node 2: If (blocks status updates)

Meta also sends "sent / delivered / read" updates to the same webhook. Without this node the workflow replies to its own status updates and floods the customer with messages.

- **Condition (Expression, you see `fx`):** `{{ Array.isArray($json.messages) && $json.messages[0]?.type === 'text' }}`

- **Operator:** Boolean → **is true**. Turn on **Convert types where required**.
- Why not “is not empty”: in this n8n version a missing value can pass that check, so receipts get through and the AI Agent fails with “No prompt specified”.
- This one condition also blocks photos, stickers and voice notes. Connect **true** to the AI Agent and leave **false** empty.
- Type the expression with no space or line break before `{{`. A leading space makes n8n read it as text, and the node fails with “Wrong type: ’ true’ is a string but was expecting a boolean”.

Node 3 (optional): AI Agent

- Prompt (User Message, Expression): `{{ $json.messages[0].text.body }}`
- Add a **Chat Model** (for example Google Gemini) and a **Simple Memory** node.
- Simple Memory → Session ID: Define below → Key (Expression): `{{ $('WhatsApp Trigger').first().json.messages?.[0]?.from_user_id ?? 'unknown' }}`. Use `from_user_id`, not `wa_id`: customers with a username have no `wa_id`, so they would all share one memory. Context window length: 5 to 10.
- Memory is erased when n8n restarts. That is normal.
- Add a clear system message that says who the business is and how to answer.

Node 4: HTTP Request (sends the reply to everyone)

A customer with a WhatsApp username has no phone number in the webhook, only an ID (`from_user_id`). The Send message node needs a phone number, so it fails for them. Meta’s send request also accepts that ID in a `recipient` field, so one HTTP Request node replies to every customer.

Create the token credential (once per client)

1. In n8n open **Credentials → Create credential → Header Auth**.
2. **Name:** `Authorization`
3. **Value:** `Bearer ` followed by the permanent token from section 6. Keep one space after Bearer and nothing after the token.
4. Save it as `WhatsApp Header Auth ClientName`.

Node settings

- **Method:** POST

- **URL** (Expression mode): `https://graph.facebook.com/v25.0/{{ $('WhatsApp Trigger').first().json.metadata.phone_number_id }}/messages`
- **Authentication:** Generic Credential Type → Header Auth → the credential above.
- **Send Body** on. **Body Content Type:** JSON. **Specify Body:** Using JSON.
- Switch the JSON field to **Expression** (you see `fx`) and paste this body:

```
{
  "messaging_product": "whatsapp",
  "recipient": "{{ $('WhatsApp
Trigger').first().json.messages[0].from_user_id }}",
  "type": "text",
  "text": { "body": {{ JSON.stringify($('AI Agent').first().json.output) }} }
}
```

Use `.first()` and not `.item` after the AI Agent, because `.item` can lose track of the trigger and turn red. Connect the AI Agent output to this node. Don't click Execute step on it, because it only works inside a real run.

The WhatsApp Business Cloud → Send message node is now optional. Keep it on the canvas, disconnected, as a backup. If you use it, the recipient must be digits only with the country code, in Expression mode: `{{ $('WhatsApp Trigger').first().json.messages?.[0]?.from }}`. Never type a fixed number there, or every reply goes to the same person.

11. Publish and connect the webhook

1. Keep ngrok **online** and n8n running.
2. In n8n, click **Publish** (the workflow must show Published, with no orange unsaved-changes dot).
3. In the Trigger node, open **Webhook URLs → Production URL**. It must start with your public address and contain `/webhook/` (not `/webhook-test/`).
4. n8n registers the webhook with Meta by itself when you publish. You don't need to fill in Meta's manual **Callback URL / Verify and save** form. That form often fails with "callback URL or verify token couldn't be validated" even when everything is right. Ignore it.
5. Check the subscription in Meta: your app → **Webhooks** (left menu, under Products). At the top, change the dropdown from **User** to **WhatsApp Business Account**. You should see your callback URL and the **messages** row set to **Subscribed**. Switch it on if needed.

All other fields can stay Unsubscribed.

12. Link the client's WhatsApp account to the app (very important)

A new WhatsApp Business Account is **not** linked to your app automatically, so Meta never sends its incoming messages to n8n. The test account is linked automatically, which is why test messages work while real ones don't. Run this in PowerShell with the permanent token (replace `YOUR_TOKEN` and `WABA_ID`):

1. Check the current state. An empty result `data {}` means it is not linked:

```
Invoke-RestMethod -Method Get -Uri
"https://graph.facebook.com/v25.0/WABA_ID/subscribed_apps" -Headers
@{Authorization="Bearer YOUR_TOKEN"}
```

2. Link it:

```
Invoke-RestMethod -Method Post -Uri
"https://graph.facebook.com/v25.0/WABA_ID/subscribed_apps" -Headers
@{Authorization="Bearer YOUR_TOKEN"}
```

It should print `success True`. Run the GET command again to confirm your app appears in the list. Don't share screenshots of the PowerShell window, because the token is visible there.

13. Test

1. From a **different phone**, send "Hi" to the client's business number. A message sent from the business number itself will not work.
2. Open the ngrok inspector (`http://127.0.0.1:4040`) and look for `POST /webhook/...`.
3. Open the n8n **Executions** tab. A new run should appear and go through the If node's **true** branch.
4. The reply should arrive in the same chat.
5. Ask a second person to test, to confirm it works for everyone (this confirms Live mode and account linking are right).

14. Troubleshooting table

Problem	Likely cause	Fix
Trigger shows red error when you click Execute	Workflow is Published and already holds the webhook subscription	Don't execute while Published. To test manually, Unpublish first
"App ID already has a webhook subscription"	Same as above, or an old stuck subscription	Unpublish, wait 10 seconds, Publish. If still stuck, Meta app → Webhooks → WhatsApp Business Account → remove subscription, then Publish again
Webhook URL shows `http://localhost`	`WEBHOOK_URL` not set in that PowerShell window	Stop n8n, set the variable, start n8n
"API access blocked" (400)	Meta refuses n8n's subscription change	Check Advanced → Allow API Access to app settings is ON. Check the OAuth credential has the right App ID and Secret
401 "Authorization failed"	Token expired or wrong, or credential from a different app	Use the permanent system user token. Check App ID, Secret and WABA ID belong to the same client
Reply loop (many "working" messages)	Trigger also fires for status updates	Add the If node (section 10) and turn the workflow off while fixing
"Invalid parameter" on Send	Empty or wrong recipient or sender	Use the expression for the recipient. Use digits with country code. Check Sender Phone Number
Messages to the real number never arrive in n8n	Account not linked to the app	Run the `subscribed_apps` POST command (section 12)
Only a few numbers get replies	App in Development mode, or the test number's 5-recipient limit	Set App Mode to Live. Have customers message the real number, not the test number

Problem	Likely cause	Fix
Error 131047 (re-engagement message)	Free-form reply outside the 24-hour window, or replying from a different number than the one messaged	Customers should message the real number. Replies are free-form only within 24 hours of their last message
Meta says number is already registered	Number still active on WhatsApp	Delete the WhatsApp account (section 4), wait, retry with Phone call verification
Meta's Verify and save fails	n8n's trigger doesn't answer that manual form the usual way	Ignore it. Rely on Publish and the Webhooks page check
`Invalid expression` in Executions	Message wasn't text (photo, sticker), or a typo in an expression	Add the type = text condition to the If node. Check expression fields
Nothing works after a PC restart	ngrok or n8n not running	Start ngrok first, then set `WEBHOOK_URL`, then n8n

15. Per-client record sheet (fill in and keep)

Item	Value
Client name	
Meta app name and App ID	
Business portfolio name	
Business phone number	
Phone Number ID	
WhatsApp Business Account ID	
System user name	
Token stored in (password manager)	

Item	Value
n8n credential names	
Public n8n address	
Production webhook URL	
Date number was registered	

16. Quick checklist for a new client

1. Create a new Meta app and a business portfolio for the client.
2. Fill in App settings → Basic, then switch to Live.
3. Free the phone number (delete its WhatsApp account).
4. Register the number in Step 2. Production setup. Note the Phone Number ID and WABA ID.
5. Create the system user, assign the app and WhatsApp account, generate a token with expiration Never.
6. Start ngrok, then n8n with `WEBHOOK_URL` (or use hosted n8n).
7. Create the OAuth credential (App ID and Secret) and the Header Auth credential (`Bearer` plus token). Create the WhatsApp API credential too only if you keep the Send message backup.
8. Build Trigger → If (strict text check) → AI Agent → HTTP Request (replies by `from_user_id`).
9. Publish. Confirm messages is Subscribed on the Webhooks page (WhatsApp Business Account).
10. Run the `subscribed_apps` POST command for the client's WABA ID.
11. Test from three phones: one with a normal number, one with a WhatsApp username, and one new customer. Check Executions after each.
12. Save everything in the record sheet. Revoke any token that was ever exposed.

17. Why some customers got no reply (WhatsApp usernames)

Some customers got no reply because they use a WhatsApp username, and WhatsApp then hides their phone number from the webhook.

What happens

- With a username, `messages[0].from` and `contacts[0].wa_id` are missing. The webhook carries only an ID, `messages[0].from_user_id` (it looks like `BD.1628392305689639`), and the `username`.
- The AI Agent still writes a good answer, but the Send message node needs a phone number. Its recipient is empty, Meta answers “Invalid parameter”, and n8n may show “Cannot read properties of undefined (reading ’replace’)”.
- Customers with a phone number work, which is why it looks random. Saving the business number as a contact also makes the phone number appear again.
- Sent, delivered and read receipts arrive at the same webhook. They have no message text, so they must be stopped at the first If node.

How to confirm the cause

1. In n8n open **Executions** and click a run where the customer got nothing.
2. Double-click the **WhatsApp Trigger** node and open the **JSON** view.
3. If `from_user_id` is there but `from` is missing, this is the cause.

The fix

Use the HTTP Request node from Node 4 for all customers. It sends to `from_user_id`, so it works with or without a phone number. Meta’s send request accepts the ID in `recipient`. If both `to` and `recipient` are sent, `to` wins.

Errors and fixes

Error or symptom	Cause	Fix
“No prompt specified” in red runs right after a reply	Sent, delivered or read receipts passed the first If	Use the strict check in Node 2 (Boolean, is true)
“Wrong type: ’ true’ is a string…”	A space before `{{`, or Convert types is off	Retype the expression with no leading space and turn on Convert types
Recipient field red, “Invalid parameter” on Send message	Customer has a username, so no phone number	Reply with the HTTP Request node using `from_user_id`

Error or symptom	Cause	Fix
“Cannot read properties of undefined (reading ’replace’)”	Same: empty recipient	Same fix
Simple Memory: “Key parameter is empty”	The key reads a field that receipts and username customers don’t have	Use `from_user_id` with a fallback value, as in Node 3
An expression turns red after the AI Agent	`.item` lost track of the trigger	Use `$('WhatsApp Trigger').first()`
“Already has a webhook subscription” when you click Execute	The workflow is Published and holds the subscription	Don’t click Execute while Published. Unpublish first to test manually
HTTP Request returns 401 or 190	Wrong token	Header Auth: Name `Authorization`, Value `Bearer ` plus the token
HTTP Request returns “Invalid parameter”	The body is not in Expression mode, or the URL is wrong	Switch the body to Expression (`fx`). The URL must end with `/<PHONE_NUMBER_ID>/messages`
Error 131047	The 24-hour window is closed	The customer must message again, then you reply

After every change

1. Click **Publish** and wait until it says Published, with no orange dot.
2. Send a message from a phone with a normal number, then from a phone with a username.
3. In **Executions**, check that the **Version** code under the title is new.
4. Expected result: one run goes If → AI Agent → HTTP Request and the reply arrives. The receipts that follow stop at the first If on the false side, with no red nodes.

Temporary workaround

If the HTTP Request node is not ready, ask the customer to save your business number as a contact and message again. Their phone number then reaches the webhook.

18. Node info (copy-paste settings for every node)

The full flow is **WhatsApp Trigger → If → AI Agent → HTTP Request**. Every expression field must be in **Expression** mode (you see `fx`), with no space or line break before `{{`. These settings cover customers with a phone number, customers with a WhatsApp username, status receipts and non-text messages.

Node 1: WhatsApp Trigger

Setting	Value
Credential	WhatsApp OAuth API (App ID and App Secret)
Trigger On	Messages
Used by Meta	Production URL (`/webhook/...`), set up automatically when you Publish

Node 2: If (only real text messages pass)

Setting	Value
Value 1 (Expression)	`{{ Array.isArray($json.messages) && $json.messages[0]?.type === 'text' }}`
Operator	Boolean → is true
Convert types where required	ON
true output	AI Agent
false output	Nothing connected

This blocks sent, delivered and read receipts, photos, stickers and voice notes.

Node 3: AI Agent

Setting	Value
Source for Prompt (User Message)	Define below
Prompt (Expression)	`{{ $json.messages[0].text.body }}`

Setting	Value
System Message	Who the business is and how to answer. Optional: add the date with `{{ $now.setZone('Asia/Dhaka').toFormat('yyyy-MM-dd HH:mm') }}`
Chat Model (sub-node)	For example Google Gemini Chat Model with its own credential
Memory (sub-node)	Simple Memory (below)
Tools (sub-nodes, optional)	Google Sheets read nodes (products, FAQ) and an append node (enquiries)

Node 3a: Simple Memory (sub-node of the AI Agent)

Setting	Value
Session ID	Define below
Key (Expression)	`{{ $('WhatsApp Trigger').first().json.messages?.[0]?.from_user_id ?? 'unknown' }}`
Context Window Length	5 to 10

The key uses `from_user_id` because customers with a username have no phone number (`from` or `wa_id`).

Node 4: HTTP Request (replies to every customer)

Setting	Value
Method	POST
URL (Expression)	`https://graph.facebook.com/v25.0/{{ $('WhatsApp Trigger').first().json.metadata.phone_number_id }}/messages`
Authentication	Generic Credential Type → Header Auth

Setting	Value
Header Auth credential	Name `Authorization`, Value `Bearer ` plus the permanent token (one space after Bearer)
Send Query Parameters	OFF
Send Headers	OFF
Send Body	ON
Body Content Type	JSON
Specify Body	Using JSON (switch the field to Expression)

Body:

```
{
  "messaging_product": "whatsapp",
  "recipient": "{{ $('WhatsApp
Trigger').first().json.messages[0].from_user_id }}",
  "type": "text",
  "text": { "body": {{ JSON.stringify($('AI Agent').first().json.output) }} }
}
```

Connect the AI Agent output to this node. Use `.first()` and not `.item` after the AI Agent.

Optional: Send message node (backup, phone-number customers only)

Setting	Value
Credential	WhatsApp API (permanent token and WABA ID)
Resource and Operation	Message, Send
Sender Phone Number	The client's number, picked from the dropdown
Recipient's Phone Number (Expression)	`{{ $('WhatsApp Trigger').first().json.messages?.[0]?.from }}`
MessageType	Text
Text Body (Expression)	`{{ $json.output }}`

This node fails for customers with a username, because they have no phone number. Don't connect it unless you add the split below.

Optional split: phone number goes to Send message, username goes to HTTP Request

Put an If node between the AI Agent and the two senders:

Setting	Value
Value 1 (Expression)	`{{ ($('WhatsApp Trigger').first().json.messages?.[0]?.from ?? '').length > 0 }}`
Operator	Boolean → is true
Convert types where required	ON
true output	Send message
false output	HTTP Request

The simpler setup is to skip this split and use only the HTTP Request node for everyone.

Optional: reply to photos and stickers

Connect the **false** output of Node 2 to an HTTP Request node with the same URL and credential as Node 4, and this body:

```
{
  "messaging_product": "whatsapp",
  "recipient": "{{ $('WhatsApp
Trigger').first().json.messages[0].from_user_id }}",
  "type": "text",
  "text": { "body": "Please send your question as a text message." }
}
```

Only do this if the false output cannot carry receipts. Receipts also go to the false side and would trigger a reply loop, so put a check before it that `messages` exists: `{{ Array.isArray($json.messages) }}` with Boolean → is true.

Expressions to avoid

Avoid	Why	Use instead
`$('WhatsApp Trigger').item` after the AI Agent	Can lose track of the trigger and turn red	`$('WhatsApp Trigger').first()`
`messages[0].from` or `contacts[0].wa_id` as a key or recipient for all customers	Missing for customers with a username	`messages[0].from_user_id`
Operator “is not empty” on a field that may be missing	A missing value can pass, so receipts reach the AI Agent	Boolean “is true” with a strict check
A space before `{{`	n8n reads it as text and the condition fails	Retype the field with nothing before `{{`